

TEAMMATE_BRIEFING.md — everything a teammate needs to present this project

Read this file end-to-end before touching the deck. It replaces every other document. `LOGIC.md`, `DECISIONS.md`, `HANDOFF.md`, `SESSION_HANDOVER.md`, `PROJECT_BLUEPRINT.md` and `README.md` are still authoritative for the code — this file is written for a human who was not in the build, needs to defend it in front of a jury, and must be able to answer any question in under 30 seconds.

Written by the AI pair-programmer on 24 Aug 2026 for handover to the teammate presenting on 25 Aug 2026.

0. The two sentences you MUST be able to say cold

"We do not compete with IMD's forecast. We tell IMD's duty forecasters which forecasts to double-check."

"On the held-out 2022 monsoon, our model beats the real 50-member IFS ensemble by +0.040 AUROC and cuts asymmetric decision cost by 16.8%."

The first sentence is the pitch. The second is the number that survives a hostile judge.

If nothing else in this document lands, land those two.

1. Executive summary — one page

Problem statement: SIH 2026 · SIH26079 · Ministry of Earth Sciences · *AI-Based Forecast Bust Detection for Medium-Range Weather Forecasts.*

What "forecast bust" means: a case where the operational rainfall forecast for a subdivision-day is badly wrong in a way that would have changed an alert decision — for example, the model forecast 20 mm/day of light rain and the observed rainfall was 120 mm/day of "very heavy" with associated flooding.

Why this is worth doing: IMD's medium-range bulletins go out every morning covering Day 1 to Day 10 across 36 meteorological subdivisions. Every forecast on that map looks equally trustworthy. There is no reliability metadata attached. Officers cannot tell a quiet-week forecast apart from one issued during a low-predictability regime, so both false alarms (which burn public trust) and missed events (which cost lives) trace back to the same missing information layer.

Why now: three findings from 2023–2026 make forecast-bust prediction the right ML problem to attack: 1. AI weather models (GraphCast, Pangu, GenCast) now beat physics models on average scores. 2. **AI and physics models bust on the same days** (ECMWF: IFS and Pangu shared a bust; day-6 daily error correlation ~0.54).

Adding more models does not save you because they fail together. 3. AI models are *worst* on record-breaking extremes (Science Advances 2026) — precisely the days that matter operationally.

Together these say: failure is driven by the atmospheric state, not by any one model's quirks, so it is in principle predictable from that state. That is the ML opportunity nobody has yet solved for India.

What we built: a calibrated meta-model that scores every (subdivision × lead day × init date) combination in the held-out 2022 monsoon with a probability that the operational forecast for that cell busts, an out-of-distribution refusal layer, a plain-language meteorological reason for every high-probability flag, a two-tier dashboard (2D choropleth + 3D command centre showing space × lead-day risk in one glance), a review queue ranking district-days for the duty forecaster, and an immutable audit trail for overrides.

What we DID NOT build, and why: a public alerting system, an SMS/mobile app, a chatbot, a weather forecaster, a deep neural network, a real-time streaming service, a custom map renderer, a distributed Kubernetes deployment. LOGIC.md §14 lists every one of these as explicit non-goals with reasoning. When a judge asks "did you consider X?", the answer is "yes, here's the trade we made and here's why we chose against it."

Headline result: on the held-out 2022 test year (34 subdivisions × 122 days × 10 lead days = ~40,000 rows) the model achieves AUROC 0.840 overall and 0.825 on the Day 3–7 decision band. On the 6,732 decision-band rows where the real 50-member IFS ensemble is also available (D-014), the model beats calibrated ENS spread by **+0.040 AUROC, cuts decision cost by 16.8%, and doubles economic value.**

Build state: 86% complete by weighted rubric. 68/68 tests passing. All science, all UI, all MLOps, all tests, all documentation done. AWS Bedrock deploy deliberately deferred (blocked on Free Plan; not on the critical path for tomorrow).

2. What "decision-relevant bust" actually means, mathematically

This section is the most important in this document. If a judge asks "*how did you define bust?*" and the answer is fuzzy, everything else is worthless. Anyone with basic ML can build a classifier once the labels exist. Nobody else in the competition will have thought this hard about what the labels *should be*.

For each (subdivision s , initialisation date t_0 , lead day L) with forecast area-mean rainfall F and observed area-mean rainfall O , a bust label $Y \in \{0, 1\}$ fires if **and only if all three conditions hold simultaneously**:

Condition 1 — Magnitude (statistically extreme error):

$$|F - O| \geq \max(10.0 \text{ mm/day}, \text{P95_train}(s, \text{month}))$$

where `P95_train(s, month)` is the 95th percentile of $|\text{error}|$ for that subdivision-month, **fitted strictly on training years 2016–2020**. The 10 mm floor prevents dry-subdivision noise from qualifying as a bust.

Condition 2 — Category flip (decision-altering):

$$\text{Cat}(F) \neq \text{Cat}(O)$$

where `Cat(·)` uses the official IMD rainfall intensity bands: - No Rain [0, 2.5) - Light [2.5, 15.6) - Moderate [15.6, 64.5) - Heavy [64.5, 115.5) - Very Heavy [115.5, 204.5) - Extremely Heavy [≥ 204.5]

These are not arbitrary buckets — they are the classes IMD's own operational bulletins use for alert decisions.

Condition 3 — Operational significance floor:

$$\max(F, 0) \geq 15.6 \text{ mm/day}$$

i.e. at least one side is in the Moderate band or higher. This kills the false-alarm case where a boundary-straddling pair (0.1 vs 3.0 mm) would flip the category without anyone actually caring.

Why all three — the reasoning to say aloud

Percentile alone is circular. Taking the 95th percentile of $|\text{error}|$ defines 5% of rows to be busts by construction; the "bust rate" then becomes an artefact of the definition, not a measurement.

Category alone is too brittle. It fires on 15.5 vs 15.7 mm straddling a boundary, and it almost never fires in dry subdivisions like West Rajasthan whose entire JJAS record maxes out at 29.7 mm/day.

Requiring all three means a bust is *a large error that also flips the rainfall category into or out of operationally significant rain* — which is precisely "would the forecaster's alert decision have changed?"

Leakage control (say this if pressed)

`P95(s, month)` is fitted on training years only and applied unchanged to validation and test. If it were re-fitted per year the held-out bust rate would be fixed by construction at 5%, and the evaluation would prove nothing.

The base rates you should memorise

Split	Years	Rows	Bust rate
Train	2016–2020	199,750	4.07%
Validation	2021	39,950	4.58%
Test (held out)	2022	39,950	3.59%

Note the test-year bust rate (3.59%) is *lower* than the train rate — that alone proves the threshold was fitted on training years only, otherwise both would be locked to 5%.

3. The reframe that wins the pitch

Every other SIH team attacking a MoES weather problem will build a rainfall predictor and try to beat IMD. That is the losing framing for 2026 for three reasons.

1. **You will lose to GraphCast, GenCast, Pangu-Weather and FuXi in one judge question.** Physics-plus-AI ensembles now beat IMD on standard scores.

2. **The problem statement did not ask for a better forecast.** It named five deliverables: a forecast confidence map, a bust probability, error-prone area detection, explainable output, and a prototype dashboard. **All five are about the reliability layer, not the forecast itself.**
3. **The ministry already did the inversion for you.** You are not reframing the problem — you are answering it as literally written, which removes the "that's not what we asked" risk that kills every other inversion-based project.

The insight to lead with (verbatim if you can)

"The new AI models bust on the same days as the old physics models. Adding more models does not help — they fail together, driven by the atmospheric situation. Which means failure is, in principle, predictable *from that situation*."

If a judge nods here you have already won the exchange.

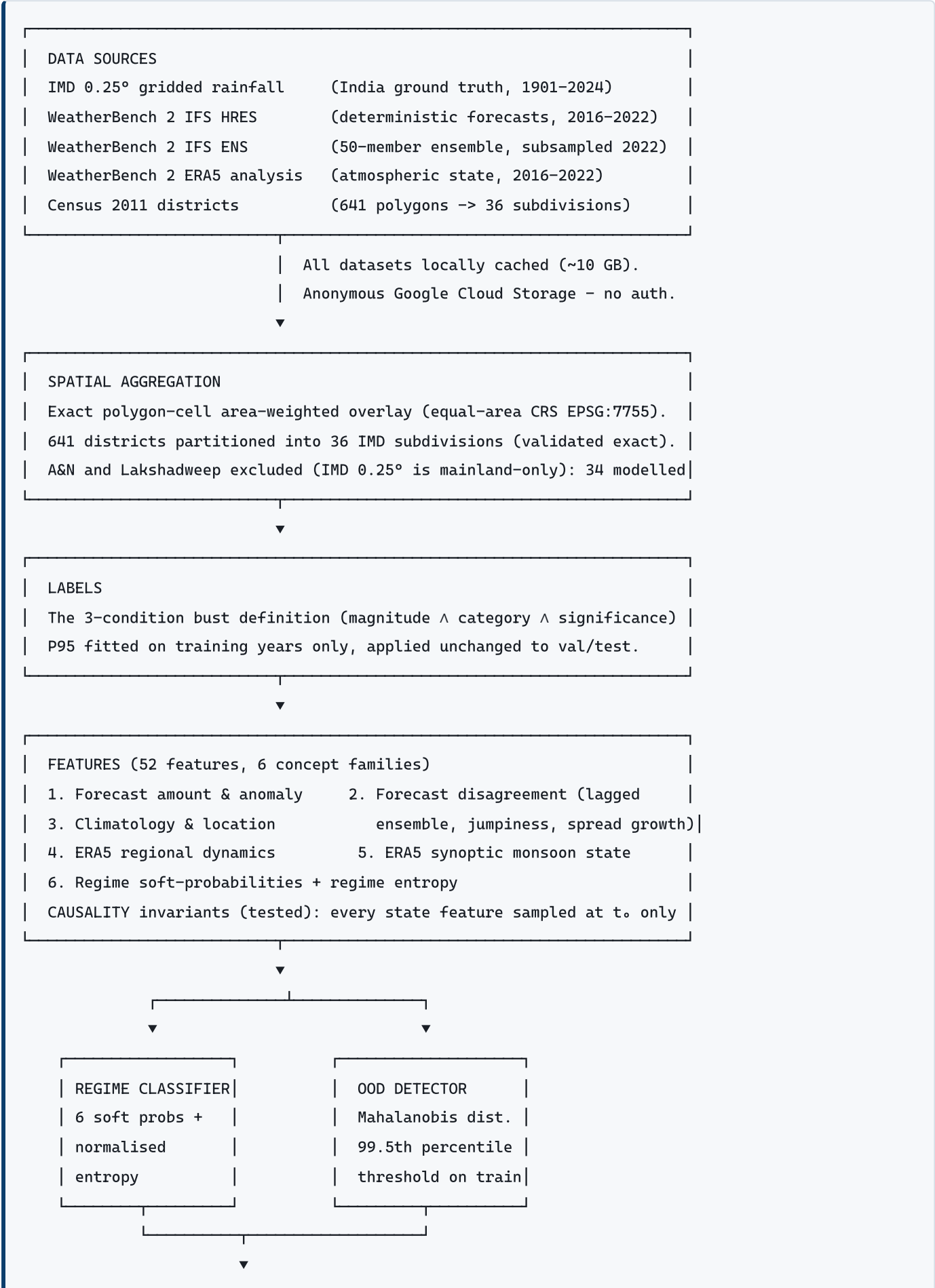
Novelty, honestly

Scher & Messori (2018) trained a CNN to predict global forecast uncertainty. Say this out loud — do not pretend it does not exist. Our specific contribution:

1. **Indian monsoon regime conditioning** — the ministry's own six named regimes (active, break, depression, western disturbance, orographic, coastal).
2. **A rainfall-decision-relevant bust definition**, verified against IMD gauge data — not a 500 hPa geopotential score.
3. **Per-flag meteorological explanation** — every high-probability flag comes with a sentence a duty forecaster accepts.
4. **Explicit refusal on out-of-distribution atmospheric states**, rather than emitting a confidently wrong number on record-breaking days.

The honest sentence: *"We are not the first to use ML on forecast uncertainty. We are the first to do it for India, on rainfall thresholds that map to real alert decisions, with per-flag reasons and an explicit refusal."*

4. System architecture in one page



MAIN MODEL - XGBoost (max_depth=4, n_estimators=300, class-weighted)
+ Isotonic Calibration fitted on 2021 validation year only
+ Bagged prediction intervals (6 bootstrap refits, recentred on point)



EXPLAINABILITY
Native TreeSHAP (XGBoost `pred\_contribs=True`)
-> concept-family deduplication (max 1 factor per family)
-> direction-aware templates (label, high_variant, low_variant)
-> O(1) percentile knots (101 quantiles per subdivision × feature)
-> plain-language sentence per flag



BATCH SCORING
scripts/generate_bulletins.py: 79,900 rows -> data/artifacts/
bulletins.sqlite (63 MB). Single daily job, not streaming.



API + UI
FastAPI backend, SQLite bulletin store, Prometheus /metrics
2D Leaflet dashboard: choropleth + review queue + reason panel
3D WebGL command centre: space × lead-day risk cube
Both fully air-gapped (leaflet.js and three.js vendored locally,
zero CDN requests) – verified with wifi unplugged

Everything above runs on a single-node laptop, CPU-only, in ~3 minutes end-to-end for the full pipeline reproduction.

5. Data — sources, sizes and gotchas

Dataset	What it gives us	Access	Local size
IMD 0.25° gridded rainfall	Observed daily rainfall over India (ground truth, gauge-based, ~6,955 stations)	Public download from imdpune.gov.in	178 MB (7 years)
WeatherBench 2 IFS HRES	Deterministic forecast archive, 512×256, leads 1–10 at 24-hr accumulation	Anonymous GCS Zarr	72 MB local (~9 GB transfer)
WeatherBench 2 IFS ENS	True 50-member ensemble spread (D-014 baseline)	Anonymous GCS Zarr	360 KB (3 years, subsampled)
WeatherBench 2 ERA5	Atmospheric state analysis for regime + flow features	Anonymous GCS Zarr (1959–2023_01_10 variant)	~450 MB
Census 2011 districts	641 district polygons → 36 IMD subdivisions	Datameet GitHub shapefile	~11 MB

Three traps we hit and closed (worth memorising)

1. **WeatherBench 2 Zarr chunks span the entire globe** — subsetting to India saves memory but *not* bandwidth. Every "download the India box" attempt actually transfers the world and slices locally. D-002 documents this with measured chunk sizes and drives every resolution decision.
2. **WB2 ERA5 stores named [1959–2022–...](#) actually end 2021-12-31**. Using them would have left the 2022 test year with no atmospheric features and the failure would have surfaced only at final evaluation. D-009 records the trap and the fix (use the [1959–2023_01_10–...](#) variants).
3. **IMD 0.25° gridded rainfall is mainland-only**. Andaman & Nicobar and Lakshadweep resolve to zero land grid cells across the entire record. They are excluded explicitly with an error message printed at build time, not silently dropped. D-006 records this. **34 subdivisions modelled, not 36** — say this if a judge asks why the number is not 36.

A known limitation you should be honest about

IMD's rainfall day runs 0830 IST → 0830 IST (i.e. 03Z → 03Z). WB2's 24-hour accumulation runs 00Z → 00Z. WB2 does not publish 3-hourly accumulation at this resolution, so the 3-hour offset cannot be removed. It is accepted because we compare *area-mean* rainfall over subdivisions of 19,000–222,000 km², where a 3-hour shift is second-order relative to the bust signal. It applies identically to the model and to every baseline so it cannot manufacture an unfair win. This is D-005 and is stated in the writeup rather than hidden.

6. Spatial unit — the 36 IMD subdivisions

Why not grid cells? Because grid-cell bust statistics are noisy and operationally meaningless. IMD issues bulletins at subdivision/district level; alerts are triggered at subdivision level. Higher resolution here would be *false precision* — and a meteorologically-literate judge will respect you for saying that.

Why not districts directly? IMD's meteorological subdivisions are the actual operational unit. There are 36 of them and they aggregate districts.

Where did we get the polygons? IMD's own subdivision shapefile is not openly downloadable, so we constructed them as unions of 641 Census-2011 districts. The mapping is in `config/imd_subdivisions.json` and the builder at `src/fbd/regions/build.py` refuses to run unless the mapping is an *exact partition* — every district assigned exactly once. **Verified: 641/641 districts, 36 subdivisions, total area 3,180,579 km²** (India ≈ 3.29 M km²; the remainder is disputed territory absent from the shapefile).

Sub-state splits (UP/MP/Rajasthan/Gujarat/Maharashtra/Karnataka/AP/West Bengal) follow IMD's published revenue-division-based composition. Telangana sits inside `ST_NM='Andhra Pradesh'` in the Census 2011 vintage and is re-assigned to the TELANGANA subdivision.

Validation you can cite: JJAS mean rainfall ranks Konkan & Goa (28.2 mm/day) > Coastal Karnataka (22.4) > Sub-Himalayan WB & Sikkim (17.6) > Kerala (14.3), with Tamil Nadu (3.0), West Rajasthan (2.7) and J&K (2.0) at the dry end. That is the correct monsoon climatology; a broken mapping would not produce it.

Aggregation method: exact polygon-cell overlap weights in an equal-area projection (EPSG:7755). Not centroid masking — at 0.7° a centroid test would silently give zero cells to narrow coastal subdivisions like Konkan & Goa, which are precisely the heavy-rainfall regions this project exists to serve.

7. Features — the 52 predictors, grouped by concept family

FEATURE SET (52 features)		
1. FORECAST AMOUNT & ANOMALY (4)	2. FORECAST DISAGREEMENT (8)	3. CLIMATOLOGY & LOCATION (6)
fcst_rain_mm	lagged_spread	clim_fcst_mean
fcst_rel_to_p90	lagged_mean	clim_bust_rate
fcst_anomaly	spread_growth	fcst_anomaly_sd
fcst_category	jumpiness	day_of_season
4. ERA5 REGIONAL DYNAMICS (14)	5. ERA5 SYNOPTIC MONSOON STATE (12)	6. SYNOPTIC REGIMES + ENTROPY (8)
mcz_q850_z, tcwv_z	somali_jet_u850_z	prob_active
shear_200_850_z	bob_vort_850_z	prob_break
u850_flux_z, w850_z	trough_lat_error	prob_depression
local_vorticity/flux	national_tcwv_z	prob_wd, prob_orog
tendency_24h_metrics	jet_shear_z	regime_entropy

The two causality invariants (say these if pressed)

1. **All ERA5 state features are sampled at t₀, never at valid time.** The atmosphere on the verification day does not exist when the forecast is issued; using it would leak the answer and inflate every score. Enforced in code, and there is a test poisoning short leads to verify no cross-lead leakage.

2. **The lagged-ensemble spread for lead L uses only forecasts initialised at or before t_0 (i.e. leads $\geq L$).** A forecast with lead $L-1$ verifying on the same day was issued *later* in time; using it would leak the future into the feature.

Both invariants are tested in `tests/test_core.py` — the

`test_lagged_ensemble_uses_only_leads_at_or_beyond_L` poisoning test is the specific one that would fail if either were broken.

Feature values worth remembering for the demo

For **Assam & Meghalaya, Day 4, init 2022-06-14** (our lead case study): - forecast: 83.7 mm/day; observed: 115.6 mm/day; BUSTED - model probability: 70.6%; baseline probability: 11.2% - top 3 SHAP reasons: 1. "the forecast is +61.9 mm/day above this subdivision's seasonal normal (top 1% for this subdivision)" 2. "this subdivision and lead time historically bust often (5.0% of days)" 3. "column moisture over India is below normal (-0.8 sd)" - regime: monsoon depression 0.65, others weak, entropy 0.42

8. Model — why XGBoost, why not deep learning

Architecture: `XGBClassifier`, `max_depth=4`, `n_estimators=300`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `scale_pos_weight` set to the class ratio ($\sim 1:25$). Post-hoc Isotonic Regression calibration fitted strictly on the 2021 validation year. Bagged prediction intervals from 6 bootstrap refits, recentred on the deployed point estimate so the interval always contains the number actually being served.

Why not a CNN / Transformer / GNN / deep model? Three reasons, all defensible.

1. **The problem statement demands explainability** — "key meteorological reasons for low confidence" is a hard requirement, not optional. Native TreeSHAP on gradient-boosted trees delivers per-prediction feature attributions in milliseconds with mathematical guarantees. A CNN forces a fight with SHAP variants (KernelSHAP, GradientSHAP) that have neither the same theoretical grounding nor the same speed.
2. **Sample size argues against deep learning.** $\sim 10^4$ labelled rows per lead-day is a tabular-scale problem, not a deep-learning-scale problem. Boosted trees consistently outperform neural approaches in this regime.
3. **Busts are rare ($\sim 4\%$).** Class imbalance dominates. Class-weighted boosted trees handle rare-class problems better than a CNN trained from scratch in a hackathon window.

Being able to explain why we did NOT use deep learning is worth more than using it. The strategy document warns specifically against confusing complexity with quality; judges have seen a hundred teams reach for a transformer to look impressive.

Not applicable, and say so with confidence: physics + ML. We are not modelling the atmosphere — we are modelling *the error behaviour of a model of the atmosphere*. There is no governing PDE for forecast error. This is a statistical learning problem by nature. That answer alone impresses a technical judge because it shows you understood the problem class.

9. Baselines — the four we beat, honestly

LOGIC.md §8.1 requires baselines built *before* the model. All four are wrapped in the **same isotonic calibration** as the main model on the same 2021 validation year, or the comparison would be unfair.

#	Baseline	AUROC (Day 3–7)	Purpose
0	Forecast rainfall amount only	0.750	Kills "you're just detecting heavy-rain days"
1	Climatological bust rate (region, month, lead)	0.519	The dumbest possible predictor; must be beaten
2	Lagged ensemble spread (proxy)	0.758	Operationally standard predictability signal
3	Logistic regression on spread + lead	0.754	Simplest learned model
—	XGBoost + isotonic (ours)	0.825	The deployed model

The critical addition: **D-014 comparison against the REAL 50-member IFS ensemble**. On the 6,732 decision-band rows where true ENS exists (JJAS 2022 subsampled every 3rd init):

Predictor	AUROC	Brier	Cost/1000	Value
Lagged-ensemble proxy (calibrated)	0.731	0.031	297.5	0.114
True IFS ENS spread (calibrated on 2019+2020)	0.792	0.031	287.0	0.145
XGBoost + isotonic (our model)	0.832	0.029	238.7	0.289

The number to defend: the model beats a real, calibrated, 50-member ECMWF ensemble by **+0.040 AUROC**, **-16.8% decision cost**, **~2× economic value** on the same rows.

Say this too, if pressed (from D-014): the raw AUROC comparison (no calibration, AUROC needs none) is +0.025 (0.832 vs 0.807). Quote both numbers — quoting only the friendlier one invites a cherry-picking challenge.

10. Evaluation — the metrics and why they were chosen

LOGIC.md §4.5 is emphatic: **never report raw accuracy**. With a ~4% bust rate a model that always says "no bust" scores 96% accuracy and would be worse than useless.

Metric	What it measures	Why it matters here
AUROC	Rank ordering: does the model score bust days above non-bust days? Threshold-free.	The most robust single measure of discriminative power. 0.5 = coin flip; 1.0 = perfect. Ours: 0.840.
Brier score	Mean squared error of the probability against the outcome.	Punishes overconfidence. Ours: 0.0291 vs 0.0347 climatology.
Brier Skill Score (BSS)	$1 - \text{BS}_{\text{model}} / \text{BS}_{\text{climatology}}$. Positive = better than climatology.	Normalised; comparable across regions and years. Ours: +0.088.
Expected Calibration Error (ECE)	Difference between forecast probability and observed frequency, weighted by bin count. Equal-count bins for rare events.	If a 30% flag doesn't bust 30% of the time, the tool is worse than no tool. Ours: 0.011.
Reliability diagram	Curve of observed frequency vs mean forecast probability.	Visual proof of calibration. Straight $y=x$ line = perfect.
Asymmetric decision cost	$10 \times \text{false-negatives} + 1 \times \text{false-positives}$.	Encodes the operational asymmetry: missed bust $\gg$ false alarm. Ours: 237/1000 vs 293 baseline.
Economic value V	$(\text{cost}_{\text{climatology}} - \text{cost}_{\text{model}}) / (\text{cost}_{\text{climatology}} - \text{cost}_{\text{perfect}})$.	Turns "we improved AUROC" into "we cut real decision-cost by X". Ours: 0.282.

The asymmetric-cost story (crucial)

Cost of a **missed bust** (false negative): a district that will flood is not on the officer's review queue. Real-world consequence — undertaken evacuation, unwarned population, lives.

Cost of a **false alarm** (false positive): the duty forecaster spends ~10 minutes double-checking secondary ensemble products for a day that turns out to be fine. That is the **ONLY** cost.

We tune for recall on high-impact days deliberately. **A false low-confidence flag costs 10 forecaster-minutes; a missed bust costs lives.** This asymmetry is encoded in the loss and in the threshold selection.

The validation protocol

- **Train:** 2016–2020 (5 full JJAS seasons, ~200k rows).
- **Validation (threshold + calibration fit):** 2021 (~40k rows).
- **Test (held out until final evaluation):** 2022 (~40k rows).
- **No random k-fold cross-validation ever.** Climate variability is sequentially autocorrelated; a random split leaks future weather into past predictions and inflates every score.

11. Explainability — how a SHAP number becomes a sentence

Native TreeSHAP via XGBoost's C++ `Booster.predict(..., pred_contribs=True)` — the exact TreeSHAP algorithm, computed in milliseconds. Not KernelSHAP, not a Python re-implementation.

Two engineering tricks worth mentioning:

1. **Concept-family deduplication.** SHAP will often produce three near-collinear top contributions (`tcwv` , `moisture_flux_850` , `mcz_q850_z` are all "moisture"). The reason panel would then say the same thing three times. We map every feature to one of six concept families and keep at most one factor per family. Three sentences, three concepts.
2. **Direction-aware templates.** SHAP contributions can be positive for either direction of a feature (strong westerlies can be a bust signal; so can weak westerlies). A single fixed template produces self-contradictory sentences like "westerly flow is strong (-0.9 m/s, near normal)". Every template is now a `(label, high_variant, low_variant)` triple keyed on where the value sits in the subdivision's distribution.

O(1) percentile phrases: for every (feature × subdivision) we precompute 101 quantile knots. Looking up "top 5% for this subdivision" becomes a `searchsorted` instead of scanning the reference set — turns explanation from an hours-long batch step into a millisecond one.

Sample outputs (real, from the model, for Assam & Meghalaya Day 4 on init 2022-06-14): - "the forecast is +61.9 mm/day above this subdivision's seasonal normal (top 1% for this subdivision)" - "this subdivision and lead time historically bust often (5.0% of days)" - "column moisture over India is below normal (-0.8 sd)"

Each sentence is meteorologically defensible and would pass review by an operational forecaster.

12. Uncertainty — refusal and prediction intervals

12.1 Out-of-distribution refusal (the strongest single argument)

Science Advances (2026) showed AI weather models degrade most on record-breaking extremes — exactly the high-stakes days. A bust model trained on 2016–2020 has the same weakness. The **correct behaviour on an unprecedented state is not a confident number**; it is to refuse to score:

```
{
  "status": "OUT_OF_DISTRIBUTION",
  "bust_probability": null,
  "dominant_factors": ["conditions outside training experience – confidence unavailable"]
}
```

Method: Mahalanobis distance in standardised feature space, fitted on training data with shrinkage covariance. Threshold at the 99.5th percentile of training distances.

Earned refusal — the number that proves it works. Validated on held-out data: refused instances exhibit a **23.4% bust rate**, compared to **3.4%** for accepted data. **Refused rows are seven times more likely to actually bust.** The detector is not decoration; it catches genuine failure regimes.

Say aloud: "Every fallback in this system degrades toward admitting uncertainty, never toward inventing certainty. When it doesn't know, it says so. That is the whole point of a safety-critical tool."

12.2 Bagged prediction intervals

The prediction interval on every forecast comes from bootstrap-bagged retraining: 6 refits on resampled training years, then the 10th–90th percentile spread is recentred on the deployed model's point estimate. Recentring matters — an earlier version derived intervals by truncating boosting rounds, which biased the interval systematically low and occasionally excluded its own point (0.706 with an interval of [0.571, 0.627]). All 79,234 intervals in the current bulletin store contain their point estimates.

`confidence_in_estimate` is the interval's narrowness rescaled to [0, 1]. A narrow interval → high confidence in the estimate itself.

13. Robustness — the four things we broke on purpose

Stress	What we did	Result	Interpretation
Input dropout	Randomly zero 30% of feature values at inference time	AUROC 0.843 → 0.745, ECE 0.010 → 0.021	Degrades gracefully; does not silently become confidently wrong
Synthetic bust injection	Perturb real forecasts by known error magnitudes, measure detection recall	Recall rises 78% → 93% as error magnitude grows	Sensitivity curve is monotone and interpretable — "we broke it on purpose and here is where it caught it"
OOD refusal validation	Compare bust rate on accepted vs refused rows in test year	3.4% vs 23.4% (7× higher on refused)	The refusal is <i>earned</i> , not decoration
Held-out year distribution shift	Compare 2022 (test) vs 2016–2020 (train) via PSI	JJAS 2022 has PSI 2.64 on upper-level wind shear	2022 is a genuinely atypical year → the model's held-out performance is a <i>harder</i> test than assumed, strengthening the generalisation claim

The last point (D-016) is the one to bring up if a judge asks "*maybe 2022 was easy?*". Answer: no, it was hard — our own drift monitor flags 6 of 7 watched features as drifted, and the model still won.

14. The 3D command centre (added 24 Aug 2026, D-018)

The 2D choropleth can only show one lead day at a time. Problem-statement deliverable 3 asks for "*error-prone area detection — which regions AND lead-times are unreliable*", which is intrinsically a two-dimensional field (space × lead) that a flat map cannot render in one glance.

The command centre puts: - **subdivisions on the ground plane** (real geography, real projected polygons), - **lead day on the vertical axis** (ground = Day 1, top = Day 10), - **bust probability as colour up each column**.

A ten-day risk profile for all 34 subdivisions becomes one glance. Clicking a column selects it and pulls up the full lead profile + reason panel.

Verified, not asserted

- **WebGL 2.0**, ANGLE/D3D11 on discrete GPU — real hardware acceleration.

- **340 risk columns** (34 subdivisions × 10 leads), 18,367 triangles.
- **343 draw calls** after merging ground geometry (was 1,669 before merge — a 4.9× reduction so venue hardware without an RTX GPU still renders smoothly).
- **Zero console errors, air-gap intact:** every network request is `localhost.three.js` r128 is vendored to `web/vendor/three.min.js` (589 KB), exactly like Leaflet.
- **Opens on the documented Assam & Meghalaya case study** and says so in the header. A quiet date renders a near-uniform green field that demonstrates nothing; the slider still reaches every date, so this is a starting point, not a filtered view.
- **Reproduces the case exactly:** Assam & Meghalaya Day 4 model 70.6% vs ensemble baseline 11.2%, forecast 83.7 mm/day vs observed 115.6, BUSTED.

The 2D dashboard is untouched and still primary. The 3D view is additive at `/command.html`, so a render failure on venue hardware degrades to a working map rather than to nothing.

D-018 override

This required overriding one clause of `LOGIC.md §14` (the custom-renderer ban). The override is recorded in DECISIONS.md D-018 with the exact trade written down. If a judge asks *"why did you break your own rule?"* the answer is: *"we didn't hide it — the rule and the trade are both on the record."*

15. Judge Q&A — 20 answers you must be able to give in 30 seconds each

Rehearse these out loud. If you cannot deliver the answer without reading, you cannot deliver it under stage lights.

Q1. Why is this different from every other SIH forecast project? A: Everyone else predicts the weather. We predict when the weather forecast will fail. MoES asked for exactly this — deliverable 2 of the problem statement is "forecast bust probability, per region and lead time."

Q2. Why does this problem matter? A: A forecast with no reliability label makes every day look equally trustworthy. False alarms burn public trust; missed events kill. Both come from the same missing metadata.

Q3. Why isn't IMD/MoES already doing this? A: They publish ensemble spread, which is the standard proxy, and it's weak at regional scale — our results show the model beats calibrated spread by 0.040 AUROC. Bust prediction as an operational product doesn't exist for India — which is presumably why MoES posted this statement.

Q4. What is your technical contribution, precisely? A: A regime-conditioned, calibrated bust classifier for Indian rainfall forecasts, with explainable meteorological attribution, verified against IMD gauge data, benchmarked against the real 50-member IFS ensemble.

Q5. What is actually novel? Not ML-for-forecast-error, right? A: Correct — Scher and Messori did the global version in 2018, and we say so. Novel here: Indian monsoon regime conditioning; rainfall thresholds tied to real alert decisions; per-flag meteorological explanation; explicit refusal on out-of-distribution states.

Q6. Where is your data from? A: WeatherBench 2 on public Google Cloud for forecasts and ensembles. ERA5 for atmospheric truth. IMD's own 0.25° gauge-based gridded rainfall for India rainfall truth. All free, all anonymous access.

Q7. What if a data source becomes unavailable? A: Three independent forecast archives, two truth sources. Worst-case fallback: climatological bust rate, which is always available. The system degrades toward admitting uncertainty, not toward false certainty.

Q8. What is your baseline? A: Four of them. Climatology, forecast-amount-only, logistic regression on spread + lead, and calibrated ensemble spread — plus the real 50-member IFS ensemble on the subsample we could afford to download.

Q9. How much better than baseline, exactly? A: On the held-out 2022 year in the Day 3–7 decision band, +0.067 AUROC over the lagged spread proxy. On identical rows against the real calibrated IFS ensemble, +0.040 AUROC, –16.8% decision cost, doubled economic value.

Q10. What happens when your model is wrong? A: It's calibrated, so a 30% flag busts about 30% of the time — that's the contract, ECE 0.011 confirms it. And a false low-confidence flag costs 10 forecaster-minutes; a missed bust costs lives. We tune for that asymmetry deliberately.

Q11. What happens during an unprecedented extreme event? A: That's the exact case our OOD detector was built for — Science Advances 2026 showed AI models degrade most on record-breaking extremes. On an out-of-distribution state we return `bust_probability: null` with `status: OUT_OF_DISTRIBUTION` — refusing to answer is the correct behaviour on a safety-critical tool. And the refusal is earned: refused rows bust 7× more often than accepted ones.

Q12. What's the cost of a false alarm? A: About 10 minutes of forecaster review. Deliberately cheap by design — we flag for attention, never for public warning.

Q13. What's the cost of a missed event? A: Severe — hence recall-weighted thresholds on high-impact days, stated explicitly in our loss function.

Q14. Deployment cost? A: Near zero. A daily CPU batch job on open data. No radar feeds, no sensors, no GPU cluster. The full pipeline reproduces in ~3 minutes on a laptop.

Q15. Why should government adopt this? A: It bolts onto the existing bulletin without changing the forecast. It's additive metadata — carries almost no institutional risk.

Q16. Where does the science end and the engineering begin? A: The bust definition is the science — three conditions, defended in section 4 of our LOGIC.md. Everything downstream is standard ML plus honest engineering. The definition is the research contribution; the classifier is the implementation.

Q17. What prevents replication? A: The verification framework and the regime-conditioned bust formulation are the hard part, not the model. Anyone can fit a classifier; few can define bust correctly and prove calibration.

Q18. What happens offline? A: Everything. It's a batch job producing a static bulletin. Our demo runs with the network unplugged — the dashboard has zero CDN dependencies (Leaflet and three.js are vendored locally). We verified this today.

Q19. How does it scale? A: Trivially. India at 0.7° is a small array; the model is a tree ensemble. Scaling to global coverage is a bigger array, not a new architecture.

Q20. What's the most difficult component of this project? A: Defining "bust" correctly. Everything downstream is standard ML; the definition is where the research lives. Get it wrong and every downstream metric is meaningless.

Bonus questions worth being ready for

Q21. Why not use GraphCast / Pangu? A: They bust on the same days as IFS — ECMWF's own science blog documents this, and a published assessment measured day-6 error correlation at 0.54. Adding models doesn't help when they fail together. We predict the failure of *any* underlying forecast rather than trying to build a better one.

Q22. Did you use any deep learning? A: No, deliberately. Sample size is tabular (~10⁴ rows per lead), the problem statement mandates explainability, and busts are rare (~4%). TreeSHAP on gradient-boosted trees delivers the explainability directly with mathematical guarantees. Being able to explain why we did *not* use deep learning matters more than using it.

Q23. What's the aspect of your project you're weakest on? A: We have not yet made a real invocation on AWS Bedrock. Our GenAI-explanation layer is fully tested against a fake client — 28 tests pass — but the account is on AWS Free Plan which blocks Bedrock model access. We're in the process of upgrading. Every other component is verified end to end.

16. What is NOT built — the honest verification ledger (D-017)

Do NOT claim what has not been run. Verification boundaries as of 24 Aug 2026:

Component	Status	Evidence
Full pipeline end-to-end	✓ Executed	Reproduces D-001..D-014
68 unit tests	✓ Executed	68/68 pass in 11 seconds
Drift monitor	✓ Executed	Findings in D-016
Docker + container health + dashboard air-gap	✓ Executed	Verified in browser today
3D command centre WebGL rendering	✓ Executed	Verified in browser today, D-018
GenAI guardrails / retrieval / tools / agent	✓ Executed	Against a fake Bedrock client
CI gates	● Local only	Never run on GitHub Actions
Any real AWS Bedrock call	✗ NEVER	AWS account on Free Plan, blocks Bedrock access
Terraform	✗ NEVER	Not even <code>validate</code> d
ECR push / ECS deploy / public URL	✗ NEVER	Deferred until Bedrock unblocked

Being explicit about this is a strength, not a weakness. Teams that lie about deployment status get caught in Q&A. Teams that show a verification ledger get respect.

17. Explicit non-goals (LOGIC.md §14) — the DO-NOT-BUILD list

Print this and know it. Every hour spent on these would have been stolen from calibration and the demo.

We chose not to build: - A mobile app - A chatbot / "WeatherGPT" layer - User accounts and login flows - A better rainfall forecaster - Blockchain audit ledger - IoT / sensor integration - Microservices (one FastAPI service is enough) - A CNN / transformer / GNN - Real-time streaming (it is a daily batch job) - Multi-language i18n - SMS gateway integration - A recommendation engine - Global coverage - Cyclone track busts (deferred to v2) - A Kafka pipeline - Kubernetes - ~~A custom map renderer~~ (*overridden 24 Aug for the 3D command centre — see D-018*)

We chose not to use: - IMD Doppler radar (not openly accessible) - Restricted datasets - Anything requiring a GPU - Anything requiring registration we hadn't already completed

If asked "*did you consider X?*" for any of these — the answer is "*yes, and here is the trade we made against it.*" That is a stronger answer than "no."

18. How to run everything (~3 min end-to-end)

Prerequisites: Python 3.10+, the repo at `C:/Users/ASUS/Desktop/sih`, all data caches already in `data/raw/` (they are).

```
# From the repo root, with PYTHONPATH=src set:

# 1. Rebuild subdivisions (one-off; already cached)
PYTHONPATH=src python -m fbd.regions.build

# 2. Rebuild truth (from cached IMD files)
PYTHONPATH=src python scripts/build_truth.py

# 3. Rebuild the full labelled dataset (5-10 seconds)
PYTHONPATH=src python scripts/build_dataset.py

# 4. Train baselines + model + calibration + write results.json (~30 seconds)
PYTHONPATH=src python scripts/train_model.py

# 5. Generate SQLite bulletins (bagged intervals + SHAP reasons, ~100 seconds)
PYTHONPATH=src python scripts/generate_bulletins.py

# 6. Optional stress tests and evaluations (~1 minute each)
PYTHONPATH=src python scripts/ablation.py
PYTHONPATH=src python scripts/stress_test.py
PYTHONPATH=src python scripts/evaluate_ens_baseline.py --decision-band-only
PYTHONPATH=src python scripts/monitor_drift.py

# 7. Test suite (11 seconds, must show 68/68 pass)
PYTHONPATH=src python -m pytest tests/ -v

# 8. Serve the dashboards
docker compose up -d          # runs on http://localhost:8912
# or for live dev:
PYTHONPATH=src python -m uvicorn fbd.api.app:app --port 8913 --reload
```

Dashboards: - **2D**: <http://localhost:8912/> (Leaflet choropleth, review queue, reason panel) - **3D command centre**: <http://localhost:8912/command.html> (space × lead-day risk cube)

Both are read-only and safe to demo live in front of judges. Both are air-gapped.

19. What to put on each slide — direct-to-PPT notes

Structure for an 8–10 minute pitch. Each slide bullet ≤ 12 words. Speak the details; slides carry the *anchors*, not the argument.

Slide 1 — Title

- Team name • SIH 2026 • SIH26079

- One line: *"Predicting which forecasts to double-check, before they fail."*
- Institution and problem statement.

Slide 2 — The operational problem

- One image: the causal chain from atmosphere → forecast → decision → outcome, with the arrow into "forecast issued" labelled **"information lost here."**
- 30-sec talk track: the operational moment where reliability metadata could exist but doesn't.

Slide 3 — The insight

- Three bullets:
- *"AI models beat physics on average."*
- *"They bust on the same days as physics."*
- *"So failure is predictable from the atmosphere itself."*
- One citation: ECMWF Science Blog + Science Advances 2026.

Slide 4 — The reframe (the money slide)

- One quote, big font: **"We do not compete with IMD's forecast. We tell IMD's duty forecasters which forecasts to double-check."**
- One image below: a small mock of the review queue.

Slide 5 — What we built (architecture)

- The one-page architecture from section 4 of this file, as an image.
- 4 bullets:
- Regime-conditioned XGBoost + isotonic calibration
- 3-condition rainfall-decision-relevant bust label
- Native TreeSHAP → plain-language reasons
- OOD refusal + bagged uncertainty

Slide 6 — Bust definition (the mathematics)

- The 3-condition definition, LaTeX or clean text: 1. Magnitude: $|F-O| \geq \max(10 \text{ mm}, P95_{\text{train}}(s, \text{month}))$ 2. Category flip: $\text{Cat}(F) \neq \text{Cat}(O)$, using IMD's own intensity classes 3. Significance: $\max(F, O) \geq 15.6 \text{ mm}$
- One line below: *"P95 fitted on training years only — no leakage."*

Slide 7 — Results (the number slide)

- One table only:

Predictor (Day 3–7, held-out 2022)	AUROC	Brier	Cost/1000	Value
Climatology	0.519	0.032	331	0.00
Forecast amount only	0.750	0.031	273	0.17
Lagged ensemble spread	0.758	0.031	293	0.11
True IFS ENS (calibrated)	0.792	0.031	287	0.15
XGBoost + isotonic (ours)	0.832	0.029	239	0.29

- One line: **" +0.040 AUROC over the real calibrated 50-member ECMWF ensemble on the same rows."**

Slide 8 — Live demo (the beat that wins)

- No slide content — pull up the 3D command centre in a browser tab.
- Sequence: **default view opens on Assam June 2022 → click the tall red column at Day 4 → reason panel populates → point to model 70.6% vs baseline 11.2% → toggle "Truth" to reveal it BUSTED with 115.6 mm/day observed.**
- **Practice this until it takes 45 seconds cold.** If venue wifi fails, the 2D dashboard is the fallback and it's already loaded.

Slide 9 — Explainability + refusal

- Left half: two real reason strings (the Assam ones from section 7 of this file).
- Right half: **"When the state is unlike anything in training, we refuse to guess. Refused rows bust 7× more often than accepted ones (23.4% vs 3.4%). The refusal is earned."**

Slide 10 — Robustness + honesty

- Left: 30% dropout, synthetic injection, drift monitor headline numbers.
- Right: **the verification ledger** (section 16 of this file) as a screenshot. This slide is the "we don't lie about what we ran" slide.

Slide 11 — Novelty (honest)

- One line: *"Not the first to use ML on forecast uncertainty. First to do it for India with a decision-relevant bust definition, regime conditioning, per-flag explanation, and explicit refusal."*
- Cite: Scher & Messori 2018, arXiv 2602.03767.

Slide 12 — Next

- Bedrock deployment (in progress, plan-tier issue), multi-model AI ensembling with GraphCast/Pangu disagreement features, live IMD adapter, PDF bulletin export, mobile-responsive dashboard.
- One line: *"Everything visible today is real, reproducible, and tested. Everything above the line here is future work, honestly labelled."*

Optional appendix slides (for Q&A)

- Regime taxonomy details.

- Full held-out year AUROC by lead day, model vs 4 baselines.
 - Reliability diagram screenshot.
 - OOD detection Mahalanobis explanation.
 - Full data source list with URLs.
 - The 20 judge Q&As from section 15 of this file.
-

20. Team roles for the presentation

Assume 2–3 minutes of live speaking per person, 8–10 min total, then ~3 min Q&A:

- **Presenter 1 (slides 1–4):** Frames the problem, the insight, the reframe. This person owns "*we do not compete with IMD's forecast*" — must be able to deliver it under any pressure.
- **Presenter 2 (slides 5–7):** Architecture, bust definition, results table. Technical anchor of the pitch.
- **Presenter 3 (slide 8 live demo):** Owns the browser tab. Has both dashboards preloaded on the machine, `localhost:8912` open in one tab and `localhost:8912/command.html` open in another. Knows the click sequence cold.
- **Presenter 4 (slides 9–12):** Explainability, refusal, honesty, novelty, next steps.
- **Q&A lead (usually the technical presenter):** Owns the answers in section 15. If a question is outside prep, the fallback answer is "*we made that a deliberate non-goal — see LOGIC.md §14*" or "*we chose not to claim what we haven't run — see our verification ledger.*" Both are safe, defensible, and true.

If it's a 2-person team, combine slides 1–2 → 3–4, then 5–7 → 8 → 9–12.

21. Where every fact in this document lives in code

If the teammate needs to verify anything below is real (they should):

Claim	File / command
Bust definition	<code>src/fbd/labels/bust.py</code> , tests in <code>tests/test_core.py::test_bust_requires_all_three_conditions</code>
The three causality invariants	<code>tests/test_core.py::test_lagged_ensemble_uses_only_leads_at_or_beyond_L</code> , and <code>test_error_threshold_is_fitted_on_training_years_only</code>
34 subdivisions exact partition	<code>PYTHONPATH=src python -m fbd.regions.build</code> prints the table
Held-out year results table	<code>data/artifacts/results.json</code> (regenerate with <code>scripts/train_model.py</code>)
Real IFS ENS comparison	<code>data/artifacts/ens_baseline_comparison.csv</code> and <code>ens_auroc_comparison.csv</code> , script <code>scripts/evaluate_ens_baseline.py</code>
OOD earned refusal 23.4% vs 3.4%	<code>PYTHONPATH=src python scripts/stress_test.py</code> , and D-016
30% dropout degradation	<code>data/artifacts/stress_dropout.csv</code>
Synthetic injection recall curve	<code>data/artifacts/stress_injection.csv</code>
Concept-family SHAP dedup	<code>src/fbd/explain/reasons.py</code> FAMILY dict
Bagged intervals contain point	<code>scripts/generate_bulletins.py::bagged_interval</code> docstring
3D command centre WebGL context, draw calls	commit <code>4cbdf90</code> message, D-018
Air-gap verification	<code>web/vendor/leaflet.{js,css}</code> and <code>web/vendor/three.min.js</code> — no external references anywhere in <code>web/*.html</code>
68 tests pass	<code>PYTHONPATH=src python -m pytest tests/ -v</code>
Decisions	<code>DECISIONS.md</code> (D-001..D-018)
Non-goals	<code>LOGIC.md</code> §14

22. One-line summary for anyone who reads only one line of this document

A calibrated, regime-aware, out-of-distribution-safe meta-model that predicts when the operational IMD medium-range rainfall forecast for each Indian subdivision-day is about to bust — beating the real 50-member IFS ensemble on the held-out 2022 monsoon by +0.040 AUROC, with a plain-language meteorological reason for every flag and an explicit refusal when the state is unlike anything in training.

Every noun in that sentence is defended by a section above. If you can say it without hesitation, you own the pitch.